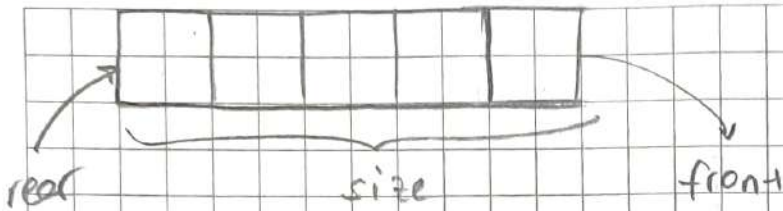


QUEUE (KUYRUK)



capacity = 50



```
struct queue{
```

```
int size, rear, front;
```

```
unsigned capacity;
```

```
int *array;
```

```
};
```

front 5 'se 4 kişi' eksilmiş

0 eleman eksilmiş olsaydı
front 1'den başlardı.

```
struct queue * createStack (unsigned capacity) {
```

```
    struct queue * yeni = (struct queue *) malloc (sizeof (struct queue));
```

```
    yeni->size = yeni->front = 0;
```

```
    yeni->capacity = capacity;
```

```
    yeni->rear = capacity - 1;
```

```
    yeni->array = (int *) malloc (sizeof (int * capacity));
```

```
    return yeni;
```

```
}
```

ör

(stack) kütlesi

A

B

C

D

S₁

S₂

S₃

A

B

C

D

S₃

S₃. push(S₁. pop())

↳ Bu şekli yapmak için push ve pop operasyonlarını yazınız.

```
void enqueue (struct queue *x, int y) {
```

```
    if (isFull(x)) {
```

```
        return;
```

```
    }
```

ERLEME

```
    x->rear = (x->rear + 1) % x->capacity;
```

```
    x->array[x->rear] = y;
```

```
    x->size = x->size + 1;
```

```
int dequeue (struct queue *x) {
```

```
    if (isEmpty(x)) {
```

```
        return;
```

```
    }
```

SILME

```
    int alinan = x->array[x->front];
```

```
    x->front = (x->front + 1) % x->capacity;
```

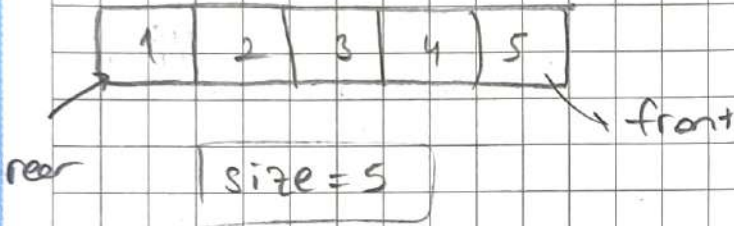
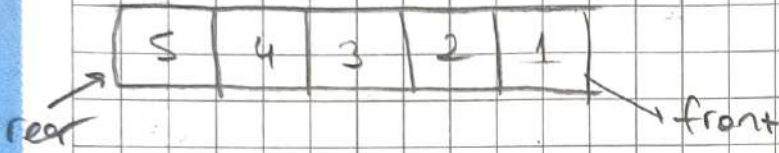
```
    x->size = x->size - 1;
```

```
    return alinan;
```

```
}
```

▽ Sıradaki gelebilir.

Öl Bir kuyruğu ters döndürmek istiyoruz.



```
# define max-size 100;
```

```
void reverse (struct queue *x) {
```

```
    int dizi[max-size];
```

```
    int top = -1;
```

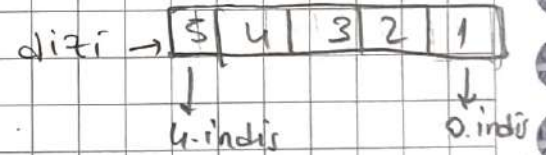
```
    while (!isEmpty(x))
```

```
        dizi[++top] = dequeue(x);
```

```
    while (top != -1)
```

```
        enqueue(x, dizi[top--]);
```

```
}
```



Ör/ Kuyruksa eleman nasıl gösterilir?

```
void display(struct queue *x) {
```

```
    int length = x->size
```

```
    while (length > 0) {
```

```
        printf("%d\t", x->array[x->front]);
```

```
        x->front = x->front + 1;
```

```
        length = length - 1;
```

```
    }
```

1. yol

X

tercih
edilmez

bu kodla
kuyruğu değiştiriyoruz
(front değişmesi)
gerek

```
void display(struct queue *x) {
```

```
    int length = x->size;
```

```
    int tmp = x->front;
```

```
    while (length > 0) {
```

```
        printf("%d\t", x->array[tmp]);
```

```
        tmp = tmp + 1;
```

```
        length = length - 1;
```

```
    }
```

2. yol

✓

tercih
edilir

bu kodda
front'u değiştirmeden
sağlıklı bir
koddan
yapıyoruz

tipi
Sayı/Veri yapısı tasarla
yin derin
struct içine ne
yazacaksın o dönü

sıralama
yapıyor

5 19 10 4 90

sort()

4 5 10 19 90

5 19 10 4 90

4 19 10 5 90

4 19 10 5 90

4 10 19 5 90

4 5 19 10 90

4 5 10 19 90 ✓

selection sort

```
void sirala(struct queue *x) {
```

```
    int i, j;    int tmp;
```

```
    int n = x->size;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = i + 1; j < n; j++) {
```

```
            if (x->array[j] < x->array[i]) {
```

```
                tmp = x->array[i];
```

```
                x->array[i] = x->array[j];
```

```
                x->array[j] = tmp;
```

```
            }
```

```
        }
```

```
    }
```

✓ Sınavda çıkabilir

ör

```
int values[10] = { 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 }
```

```
stack <int> s;
```

```
for (int i = 0; i < 10; i++) {
```

```
    s.push(values[i]);
```

```
}
```

```
int n = 25;
```

```
for (int i = 0; i < 4; i++) {
```

```
    n += s.top(); s.pop();
```

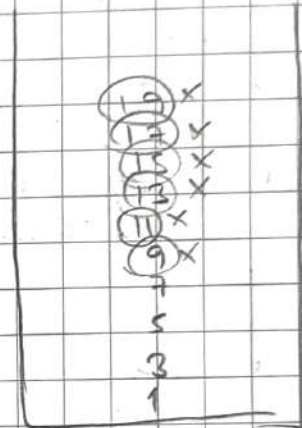
```
}
```

```
for (int i = 0; i < 2; i++) {
```

```
    n -= s.top(); s.pop();
```

```
}
```

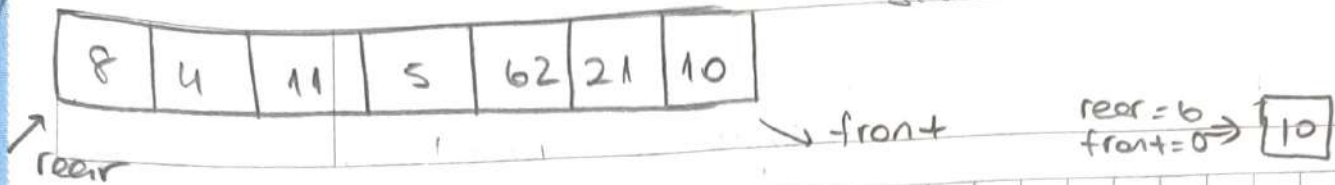
```
cout << n << endl;
```



~~n = 25~~ n = 89

Ortanca elementi bulalım.

size = 7



```
printf("%d\n", median(x));
```

```
struct queue {
```

```
    int rear, front, size;
```

```
    unsigned capacity;
```

```
    int * array;
```

```
};
```

```
void median (struct queue *x) {
```

```
    if (isEmpty(x)) {
```

```
        return;
```

```
    }
```

```
    int uzunluk = 0;
```

```
    int indis = x->front;
```

```
    int tmp;
```

```
    for (i = 0; i < uzunluk - 1; i++) {
```

```
        for (j = i + 1; j < uzunluk; j++) {
```

```
            if (array[j]) {
```

```
                x->array[i]
```

```
            }
```

```
            tmp = x->array[j];
```

```
            x->array[i] = x->array[j];
```

```
            x->array[j] = tmp;
```

```
        }
```

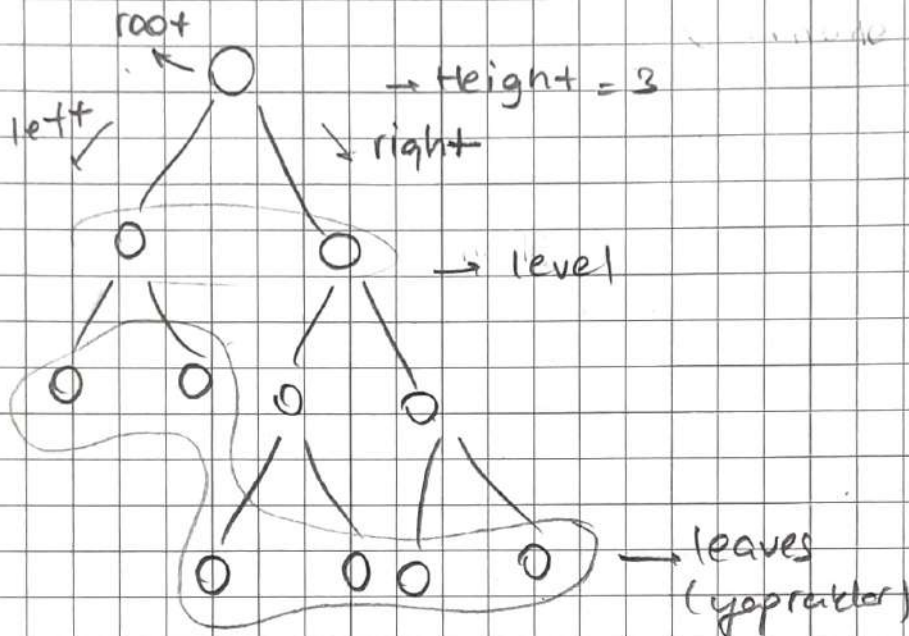
```
    }
```

```
    indis = uzunluk / 2;
```

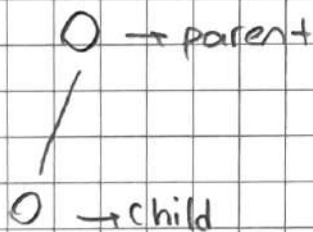
```
    printf("%d\n", x->array[indis]);
```

Binary Tree

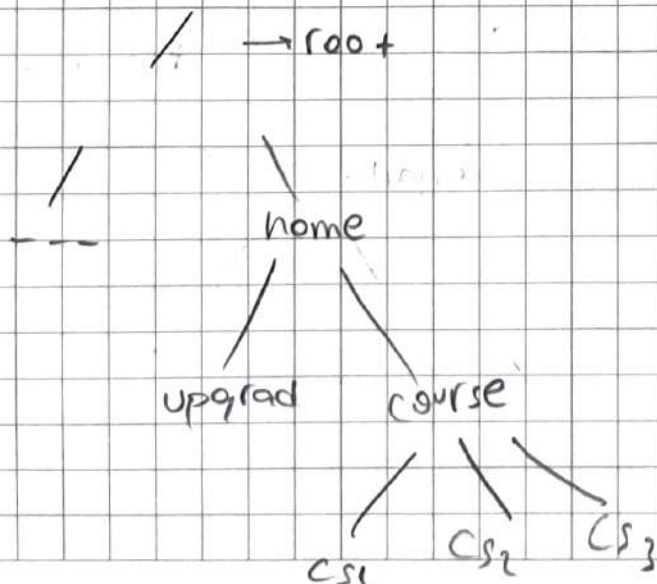
Ağaç: Hiyerarşik yapıdır. Graflara benzerdir.



alt düğümü olmayan düğümler



file system:



Neden Ağaçlar?

1. Dosya sisteminde hiyerarşik veri tutar.
2. Ortalama erişim /-arama sağlar.
3. Bağlı Listeden hızlı, diziye yavaştır.
4. Ortalama ekleme, silme sağlar.
5. Sınırsız eleman eklenebilir.

Uygulama Alanları

1. Hiyerarşik veri yönetimi
2. Bilgiyi erişim için hızlandırmak
3. Sıralı veri ile çalışırken
4. Dijital resimleri sıkıştırmak için
5. Router Algoritmaları
6. Çok aşamalı karar

Soyut Veri Tipi Olarak Nasıl Gösteriliyor?

```
struct node {
```

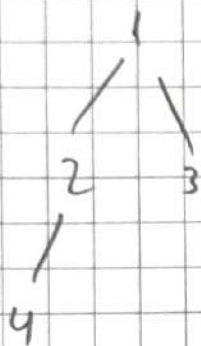
```
    int data;
```

```
    struct node * left;
```

```
    struct node * right;
```

```
};
```

tree:



1'in left'i 2

1'in right'i 3

2'in left'i 4

2'in right'i NULL

3'in left'i NULL

3'in right'i NULL

```
#include <stdio.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *left;
```

```
    struct node *right;
```

```
};
```

```
struct node *createNode(int x) {
```

```
    struct node *yeni = (struct node *) malloc(sizeof(struct node));
```

```
    yeni->data = x
```

```
    yeni->left = yeni->right = NULL;
```

```
    return(yeni);
```

```
}
```

```
void main() {
```

```
    struct node *tree = createNode(1);
```

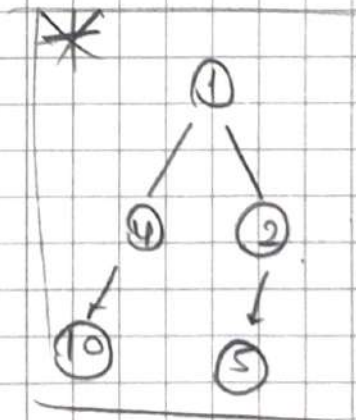
```
    tree->left = createNode(4);
```

```
    tree->left->left = createNode(10);
```

```
    tree->right = createNode(2);
```

```
    tree->right->left = createNode(5);
```

```
}
```



İkili Ağaç Tipleri

⇒ Full Binary Tree

Tanım: Her düğümü 2 veya 0 düğüme sahip (child) ağaçtır.

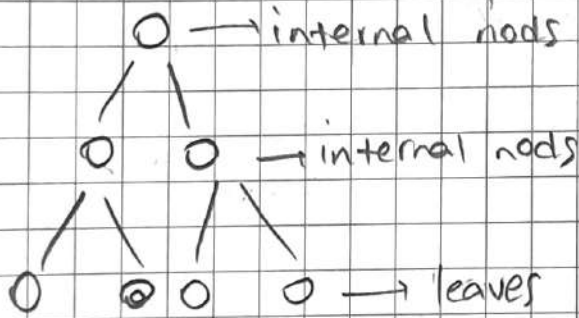


$$L = I + 1$$

Leaves

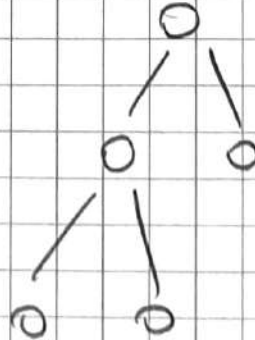
Internal
Nodes

yaprak olmayan
düğümlere verilen
addır



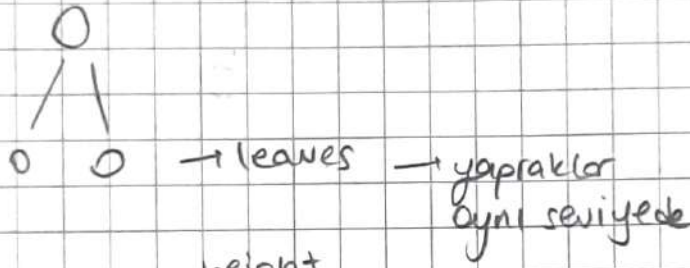
⇒ Complete Binary Tree

Tanım: Son seviye hariç tüm seviyeleri dolu ağaçtır.



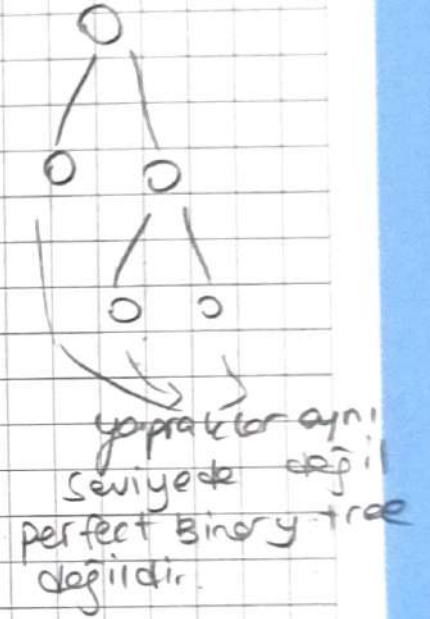
Perfect Binary Tree

Tanım: Tüm iç düğümlerin 2 adet alt düğümü vardır ve tüm yapraklar aynı seviyededir.



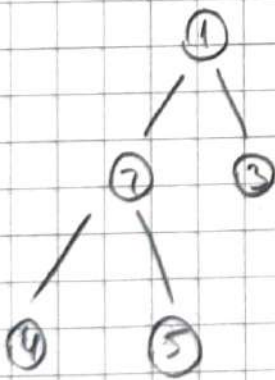
$$N = 2^h - 1$$

node

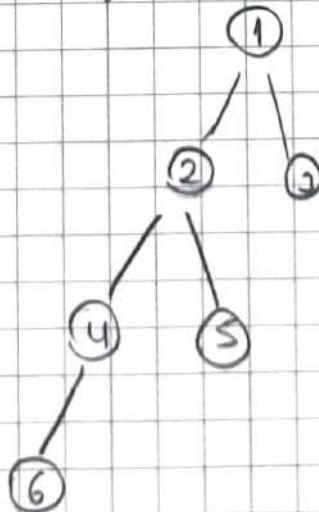


Balanced Binary Tree

$H = O(\log n)$ olan ağaçtır. ($n \rightarrow$ düğüm sayısı)



$$\log 5 = x$$
$$2^2 \rightarrow h$$



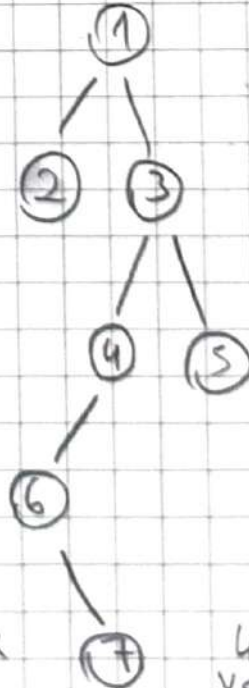
$$\log 6 = x$$

$$2^x = 6$$

$$x = 2 \rightarrow h$$

$$2 \neq 3 \text{ old.}$$

dengeyi ağaç değil.



$$\log 7 = x$$

$$2^x = 7$$

x=2

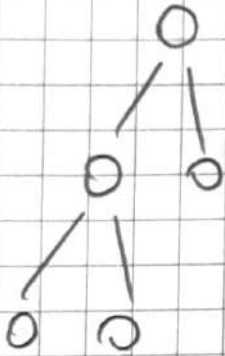
$$4 \text{ seviye}$$

var $h=4$

$4 \neq 2$ old. dengeyi ağaç değil.

AVL

Tanım: İki alt ağacın arasındaki yükseklik farkı $\max 1$ olan ağacdır.



$$2 - 1 = 1$$

AVL ✓

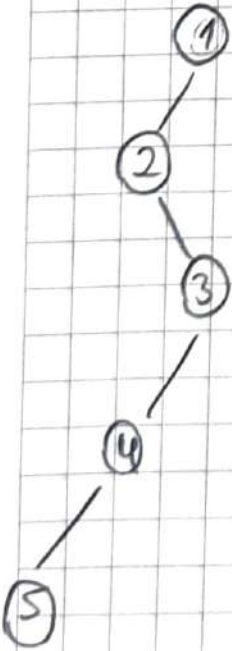


$$3 - 1 = 2$$

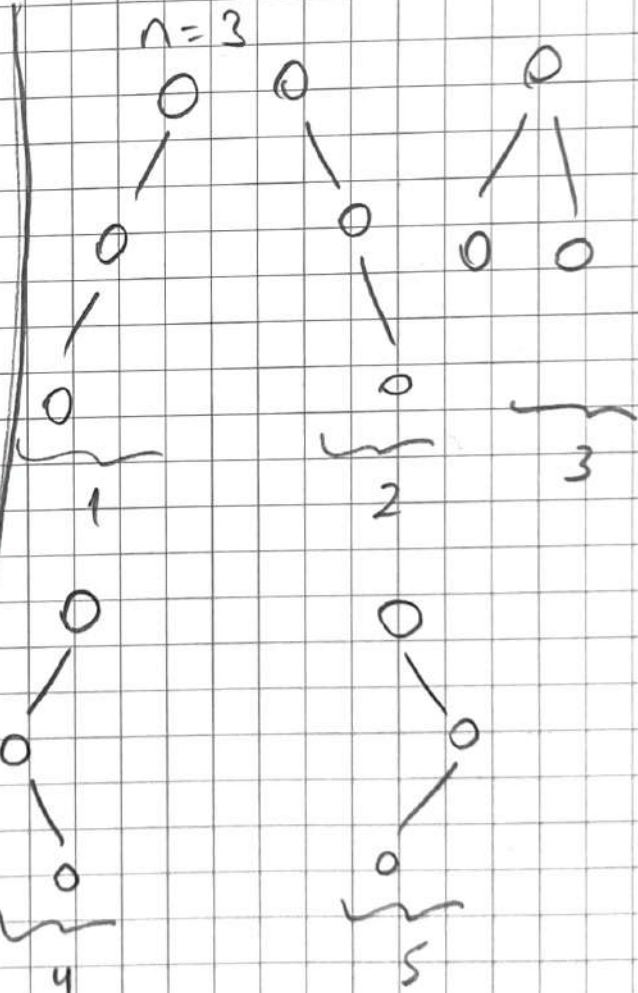
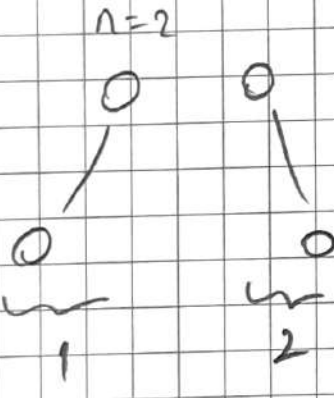
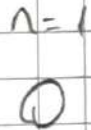
AVL ✗

Pathological Tree

Tanım: Her bir düğümün 1 adet alt düğümü olan ağacdır.



Unlabeled (Etiketless Agar)



$$T(0) = 1$$

$$T(1) = 1$$

$$T(2) = 2$$

$$T(3) = 5$$

$$T(3) = \underbrace{T(0) * T(2)}_2 + \underbrace{T(1) * T(1)}_1 + \underbrace{T(2) * T(0)}_2 = 5$$

devon

devan

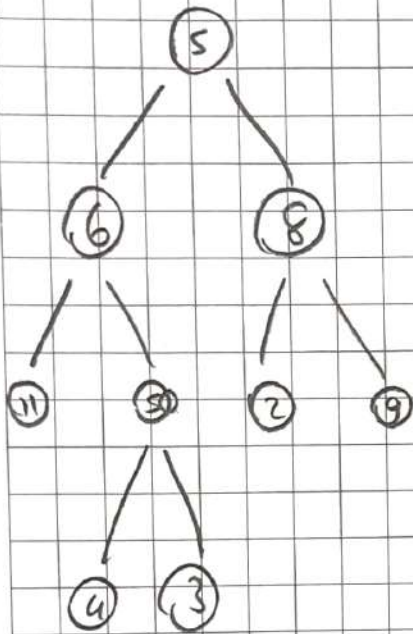
genel formül = $T(n) = \sum_{i=1}^n (i-1) \cdot T(n-i)$

$$T(n) = \frac{2n!}{(n+1)! \cdot n!}$$

Etiketli iain formüle $n!$ carpılarak geliyor.

$$T(n) = \frac{2n! \cdot n!}{(n+1)! \cdot n!} //$$

Ağacın Gezinti Türleri



Inorder → 11, 6, 4, 50, 3, 5, 2, 8, 9

Preorder → 5, 6, 11, 50, 4, 3, 8, 2, 9

Postorder → 11, 4, 3, 50, 6, 2, 9, 8, 5

Inorder

Preorder

Postorder

Gezinti
Tipleri

İçinde

Inorder → sol, kök, sağ

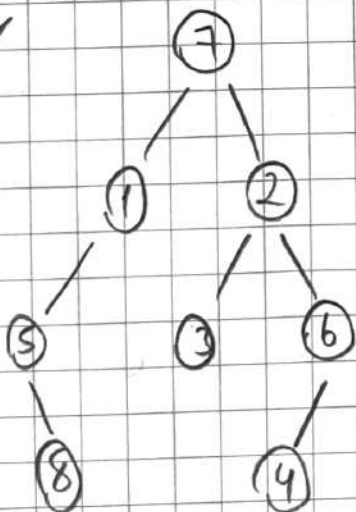
Önce

Preorder → kök, sol, sağ

Sonra

Postorder → sol, sağ, kök

ör

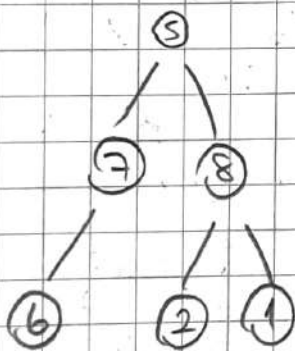


Inorder = 5, 8, 1, 7, 3, 2, 4, 6

Postorder = 8, 5, 1, 2, 4, 6, 2, 7

Inorder

```
void inorder (struct node *x) {  
    if (x == NULL)  
        return;  
    inorder (x->left);  
    printf ("%d\t", x->data);  
    inorder (x->right);  
}
```



inorder(yeni);

yeni = newNode(5);

yeni->left = newNode(7);

öğün
olusturuyor

printf(6)
inorder(NULL)
printf(7)
inorder(NULL)
printf(5);
inorder(8)

6, 7, 5, 2, 8, 1

↓

printf(2)
inorder(NULL)
printf(5)
inorder(1)



printf(1)
inorder(NULL)

Preorder

```
void preorder (struct node *x)
```

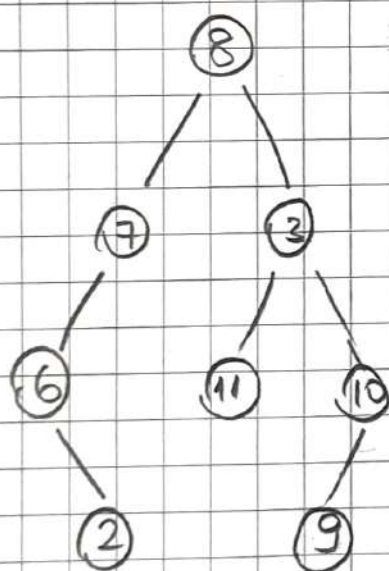
```
    if (x == NULL)
        return;
```

```
    printf ("%d\t", x->data);
```

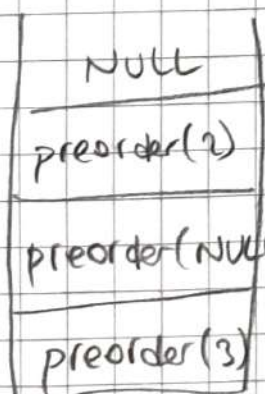
```
    preorder (x->left);
```

```
    preorder (x->right);
```

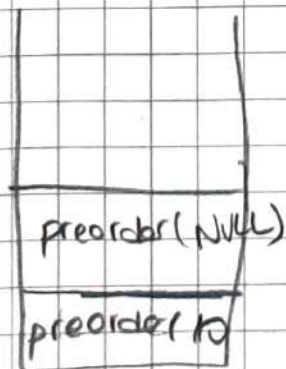
```
}
```



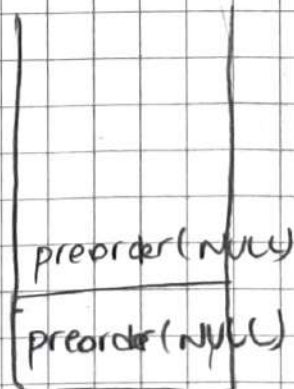
8, 7, 6, 2, 3, 11, 10, 9



preorder(7)
preorder(6)
preorder(NULL)



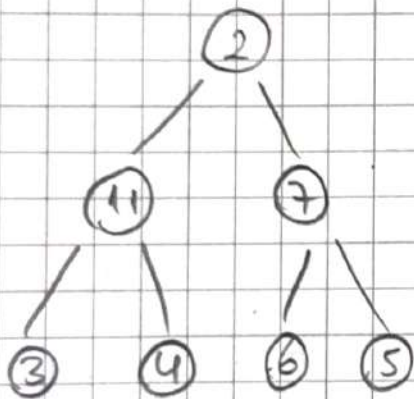
preorder(3)
preorder(11)
preorder(NULL)



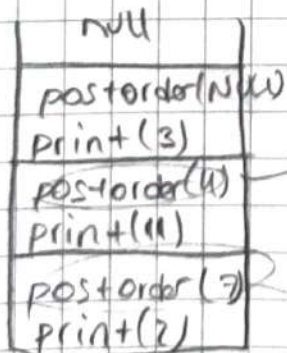
preorder(10)
preorder(9)
preorder(NULL)

Postorder

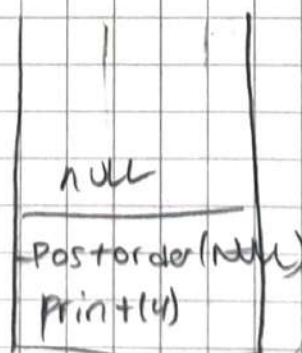
```
void postorder (struct node * x) {
    if (x == NULL)
        return;
    postorder (x->left);
    postorder (x->right);
    printf("%d\t", x->data);
}
```



3, 4, 11, 6, 5, 7, 2



postorder(11)
postorder(NULL)



postorder(7)
postorder(6)
postorder(NULL)



postorder(2)
postorder(11)
postorder(3)
postorder(NULL)

